

Document Technique

1. Introduction	3
1.1 Contexte du projet	3
1.2 Objectifs principaux.....	3
1.3 Enjeux et problématiques.....	3
1.4 Périmètre du projet	3
2. Spécifications Fonctionnelles.....	4
2.1 Cas d'utilisation	4
2.2 Fonctionnalités principales	4
2.3 Exigences fonctionnelles	5
2.4 Exigences non-fonctionnelles.....	5
3. Architecture du Système	5
3.1 Vue d'ensemble de l'architecture	5
3.2 Technologies utilisées.....	6
3.4 Sécurité	7
5. Workflow de développement (Gitflow).....	7
5.1 Les branches utilisées	8
6. Diagramme de classe	10
7. Interface utilisateurs.....	11
8. Problème rencontrés	14

1. Introduction

1.1 Contexte du projet

Le projet RareCaching fusionne le géocaching et les NFT. Il propose une plateforme ludique (Web/App) où les utilisateurs peuvent chercher des caches en réalité augmentée ou sur des cartes numériques pour obtenir des NFT. Les utilisateurs peuvent être chasseurs ou organisateurs, ces derniers créant des caches avec leurs propres NFT.

Deux modes de jeu sont proposés :

- Geo : déplacement physique et chasse avec des appareils connectés.
- Map : chasse exclusivement sur carte numérique.

1.2 Objectifs principaux

- Créer une expérience immersive de chasse aux trésors en combinant géocaching et NFT.
- Offrir un système de rareté des NFT pour favoriser la collection.
- Développer une économie participative où les utilisateurs créent leurs propres chasses.
- Monétiser via une place de marché NFT avec des micro-commissions sur les transactions.

1.3 Enjeux et problématiques

- **Expérience utilisateur** : fournir une interface intuitive et immersive.
- **Blockchain** : intégration fluide des NFT avec la blockchain.
- **Géolocalisation** : précision pour les caches en mode Géo.
- **Scalabilité** : gestion de nombreuses caches et utilisateurs tout en maintenant de bonnes performances.
- **Sécurité** : garantir des transactions sécurisées pour les utilisateurs.

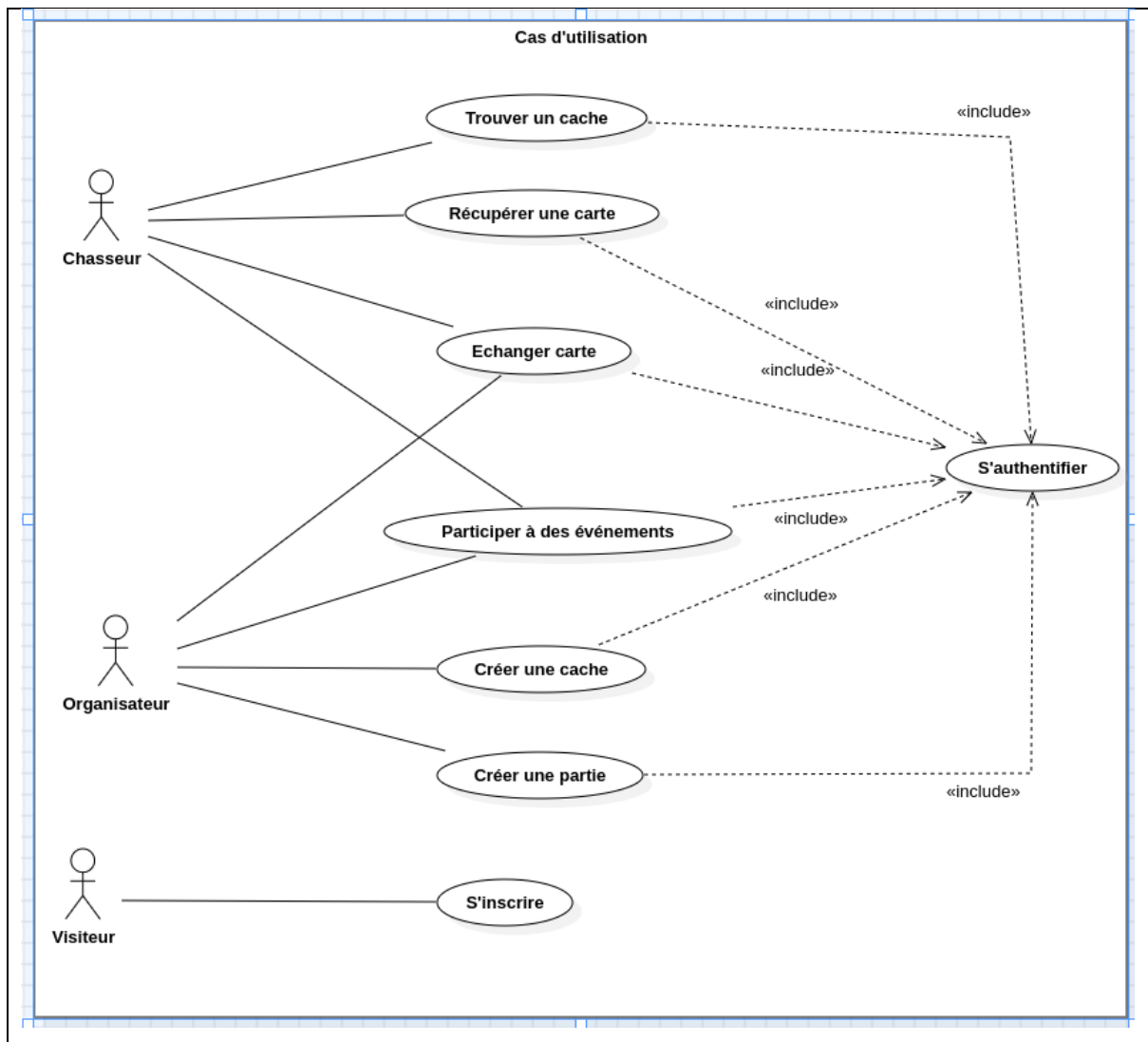
1.4 Périmètre du projet

- **Fonctionnalités couvertes** : création et découverte de caches, gestion des NFT, place de marché pour les transactions.

- **Limitations** : seuls les NFT émis sur la plateforme sont supportés, avec un déploiement initial sur la blockchain Ethereum.

2. Spécifications Fonctionnelles

2.1 Cas d'utilisation



2.2 Fonctionnalités principales

- Recherche et découverte de caches via réalité augmentée (RA) ou carte numérique.
- Création et gestion des caches par les organisateurs.
- Utilisation de cartes comme récompenses.

2.3 Exigences fonctionnelles

- Capacité à géolocaliser l'utilisateur et afficher des objets en RA.
- Algorithme de distribution de cartes basé sur leur rareté.
- Système de gestion de comptes utilisateurs.
- Système de création et de gestion des récompenses.

2.4 Exigences non-fonctionnelles

- **Performance** : L'application doit être réactive, en particulier pour l'affichage en RA et la géolocalisation.
- **Sécurité** : Les transactions NFT et les données des utilisateurs doivent être sécurisées.
- **Compatibilité** : L'application doit être compatible avec les principales plateformes mobiles (iOS et Android).

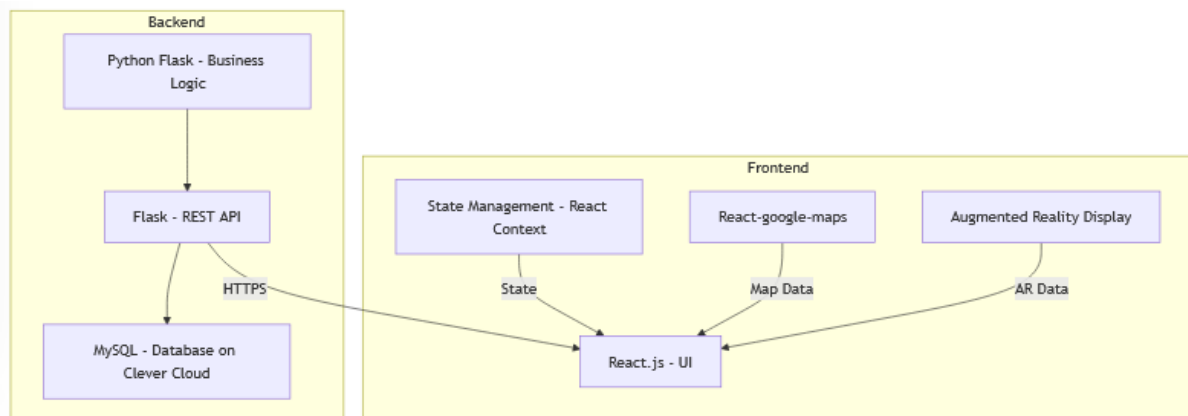
3. Architecture du Système

3.1 Vue d'ensemble de l'architecture

L'architecture du projet RareCaching repose sur une architecture client-serveur bien définie, avec une séparation entre le frontend (interface utilisateur) et le backend (gestion de la logique métier et des données).

- **Frontend** : L'interface utilisateur est développée en React.js, offrant une expérience interactive et réactive aux utilisateurs. Le frontend permet la gestion des caches en mode Geo (réalité augmentée) et en mode Map (cartographie numérique). Il communique avec le backend via des appels API (HTTP).
- **Backend** : Le backend est développé en Python Flask, un micro-framework léger et efficace pour la création d'API REST. Il est responsable de la gestion des utilisateurs, des caches, et des interactions avec les cartes. Le backend reçoit les requêtes du frontend, traite les données et interagit avec la base de données.
- **Base de données** : Le projet utilise une base de données MySQL hébergée sur Clever Cloud pour stocker toutes les données relatives aux utilisateurs, caches, et

transactions NFT. Clever Cloud offre une scalabilité et une gestion simplifiée de la base de données.



3.2 Technologies utilisées

Frontend :

- React.js pour la gestion de l'interface utilisateur et l'interactivité.
- Framework JavaScript pour construire des interfaces utilisateurs réactives.
- Utilisation de **React Router** pour la gestion des routes de l'application et de **Context API** pour la gestion globale de l'état.
- Fetch() pour les appels API au backend Flask.

Backend :

- Flask (Python)
- Framework minimaliste pour créer des API RESTful.
- Utilisation de **Flask-Login** pour la gestion des sessions et de l'authentification des utilisateurs.
- Gestion des bases de données avec **SQLAlchemy**, un ORM qui permet de manipuler MySQL via des objets Python.
- **Flask-Migrate** pour la gestion des migrations de la base de données.

Base de données :

- Hébergée sur **Clever Cloud**, elle stocke toutes les informations relatives aux utilisateurs, parties, collections, et cartes.

- Utilisation de **SQLAlchemy** pour interagir avec la base de données en tant qu'ORM, et gestion des requêtes SQL complexes

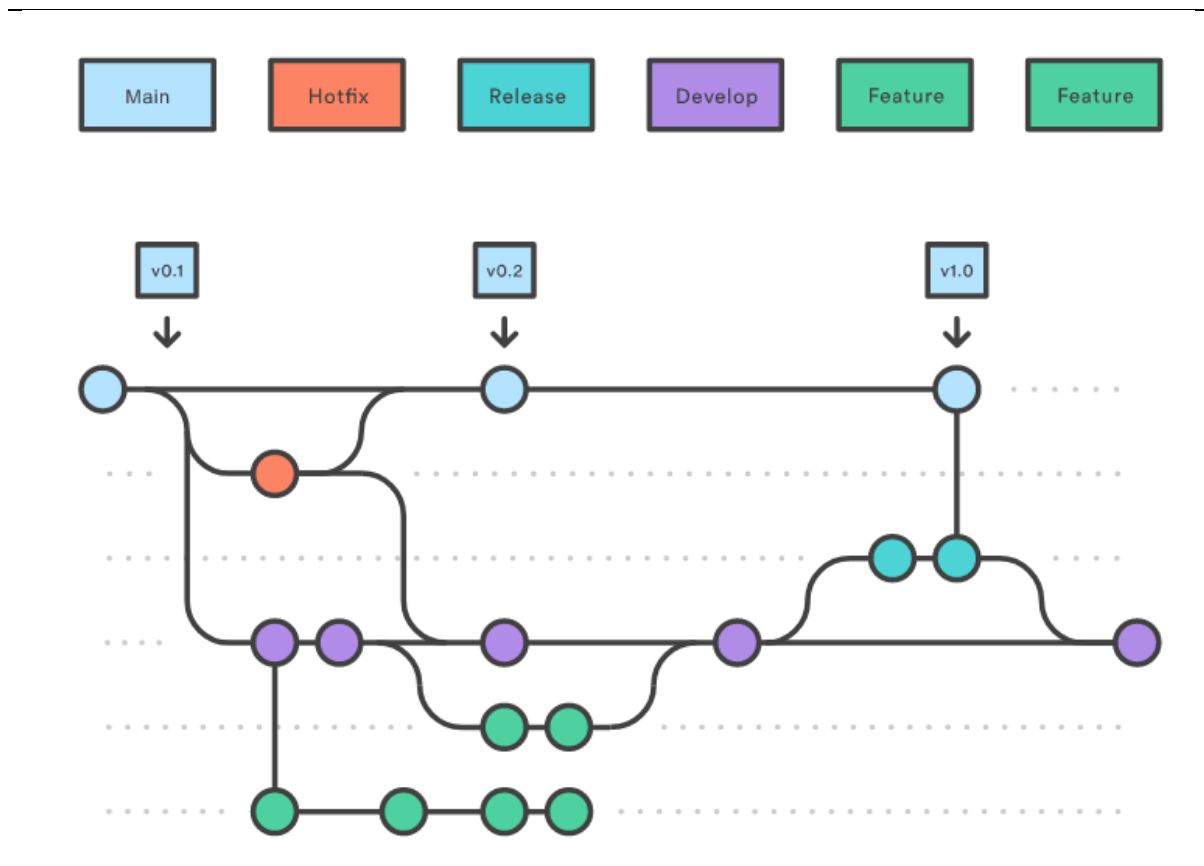
3.4 Sécurité

- **Authentification :**
 - Les utilisateurs s'authentifient via **Flask-Login**. Les sessions sont sécurisées avec des cookies.
- **Chiffrement :**
 - Les mots de passe sont hashés avant d'être stockés dans la base de données, garantissant ainsi la sécurité des mots de passe.
- **Protection des API :**
 - Les routes sensibles sont protégées par un middleware qui vérifie que l'utilisateur est authentifié avant d'accéder aux ressources.
- **Prévention des failles :**
 - Le projet suit les bonnes pratiques de sécurité pour éviter les attaques comme les injections SQL (grâce à l'ORM SQLAlchemy), les failles XSS, et les attaques CSRF (Cross-Site Request Forgery).

5. Workflow de développement (Gitflow)

On a choisi d'utiliser Gitflow pour les raisons suivantes:

- Permettre un déploiement et une intégration continue avec Heroku
- Pour garantir une application fonctionnelle et capable d'accepter des changements pendant sa période de vie
- Mise en place de processus de testing et qualité de code automatique



5.1 Les branches utilisées

- Main
 - La branche déployée
 - Règle de nommage: X.Y.Z
 - X: version majeure, changée suite à de grands changements
 - Y: version mineur, changée suite à des changements moins conséquents (ajout d'une fonctionnalité, changement de design / visuels...)
 - Z: patch, changée suite à des correctifs ou des hotfixes
- Develop
 - Branche de développement générale
 - Règle de nommage: version future de la main - date de release - titre de la feature / changement
- Feature

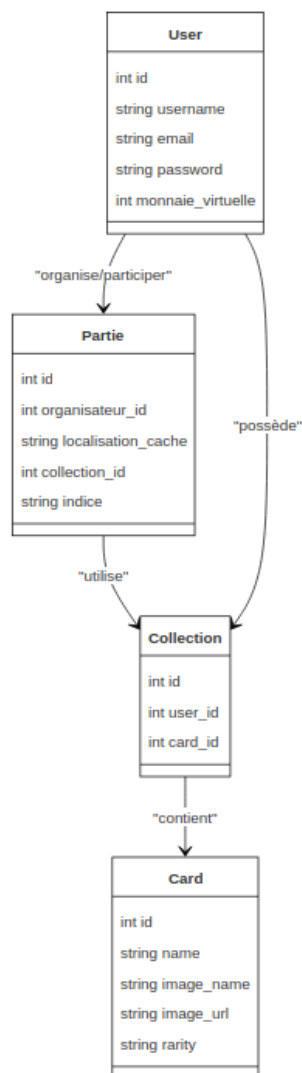
- Branche de développement d'une fonctionnalité spécifique
- Règle de nommage : Titre - Date

Occasionnellement, on utilise une branche Hotfix lié à la main pour réparer des problèmes majeurs en prod

- Hotfix:
 - Branche créé en cas de bug en prod, contient le correctif
 - Règle de nommage:
 - Hotfix / Documentation - numéro de l'issue - date
 - Exemple : 'Hotfix - 13 - 03/04

6. Diagramme de classe

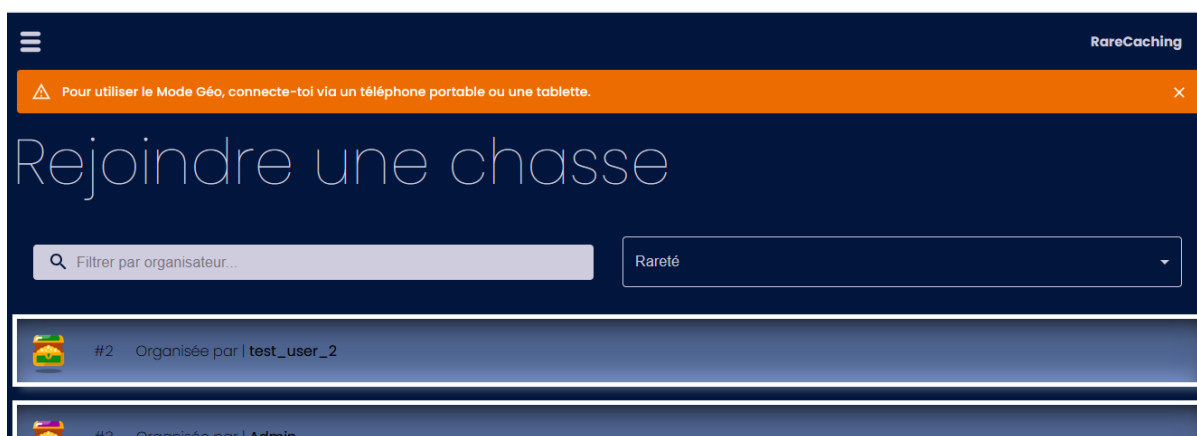
Le diagramme de classes montre que chaque **Utilisateur** possède des **Collections**, qui contiennent des **Cartes**. Les utilisateurs participent à des **Parties** et utilisent leurs collections. Chaque partie est liée à des **Indices** pour guider les joueurs. Cela organise les interactions et assure une bonne gestion des données.



7. Interface utilisateurs

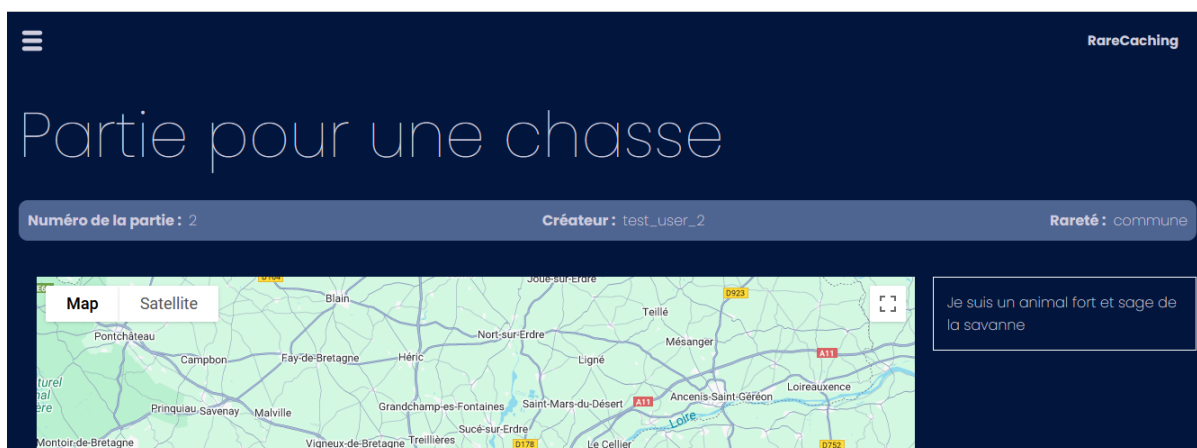
1. Rejoindre une partie / liste de parties

Permet aux utilisateurs de voir et de rejoindre des parties de chasse. Un champ de recherche et des filtres par organisateur et rareté sont disponibles. Un message en haut de l'écran recommande l'utilisation d'un mobile ou d'une tablette pour le mode jeu.



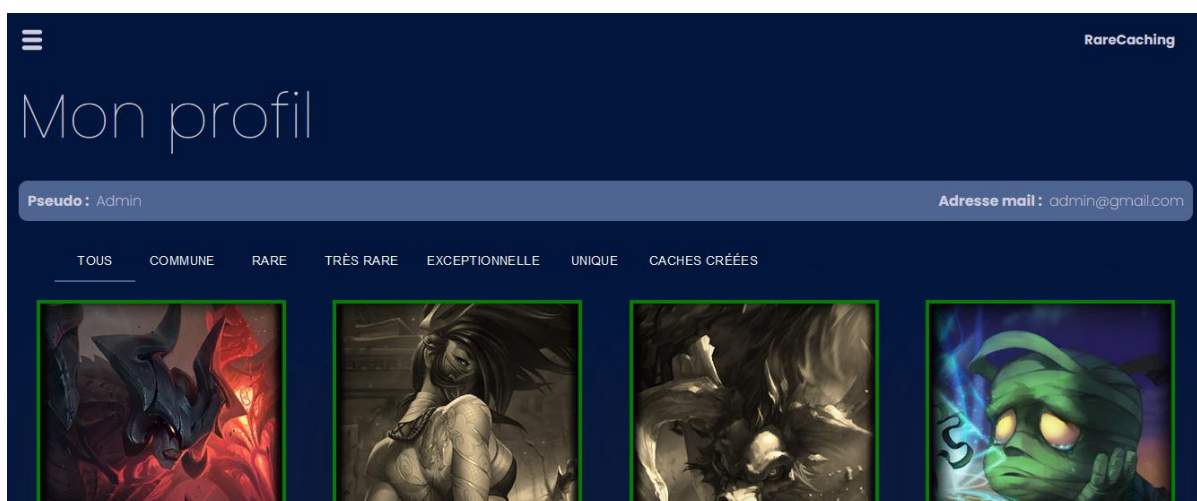
2. Ecran de jeu

Affiche les détails d'une partie spécifique, incluant le numéro de partie, le créateur et la rareté. Une carte interactive permet aux joueurs de visualiser leur environnement, avec un espace pour des indices.



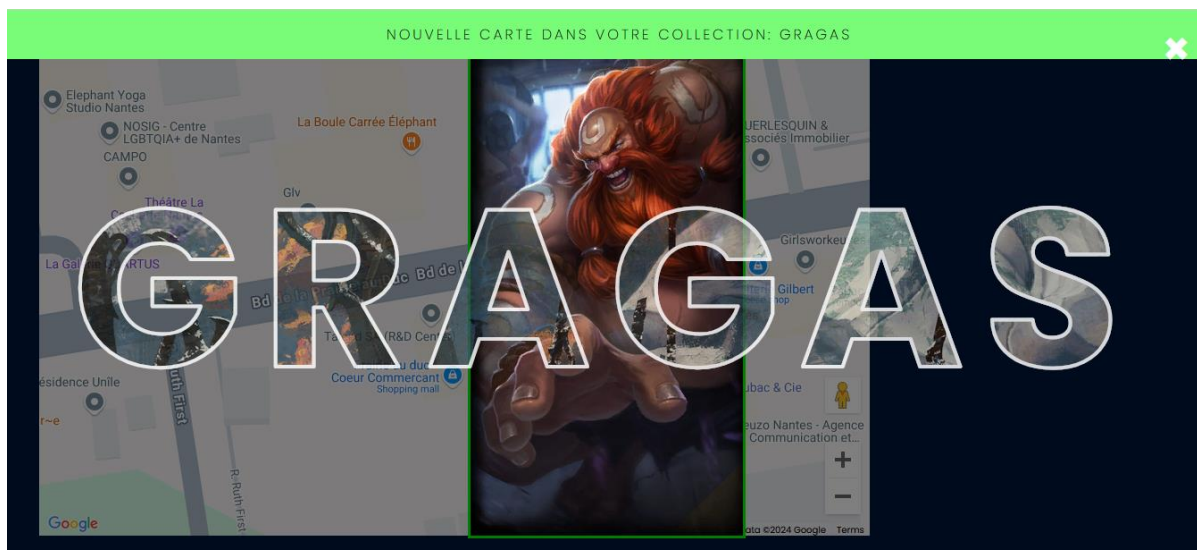
3. les cartes

Cette partie affiche la liste des cartes



4. Ecran de la victoire

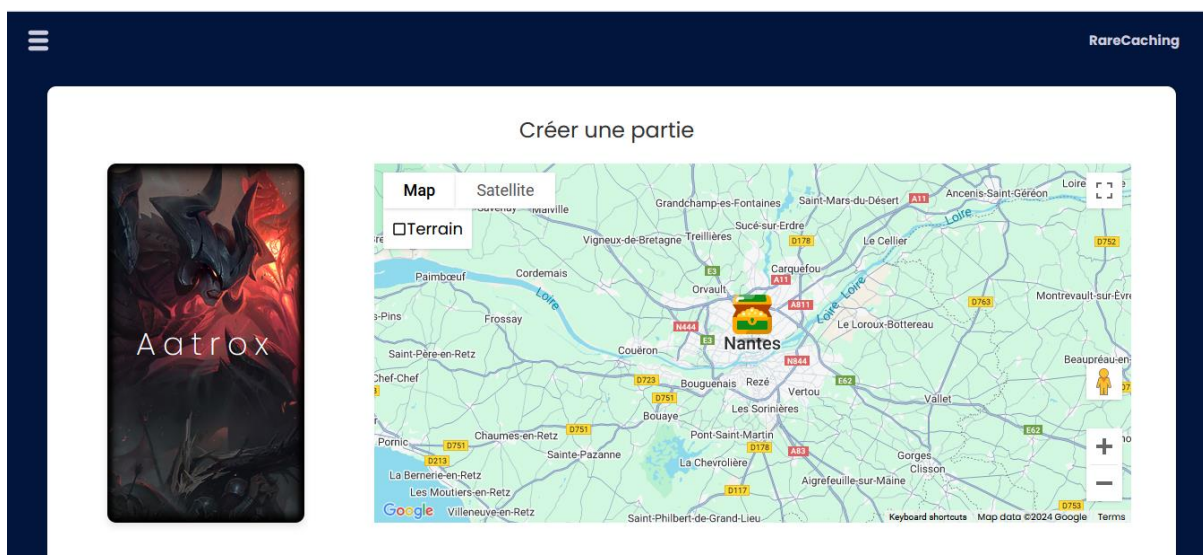
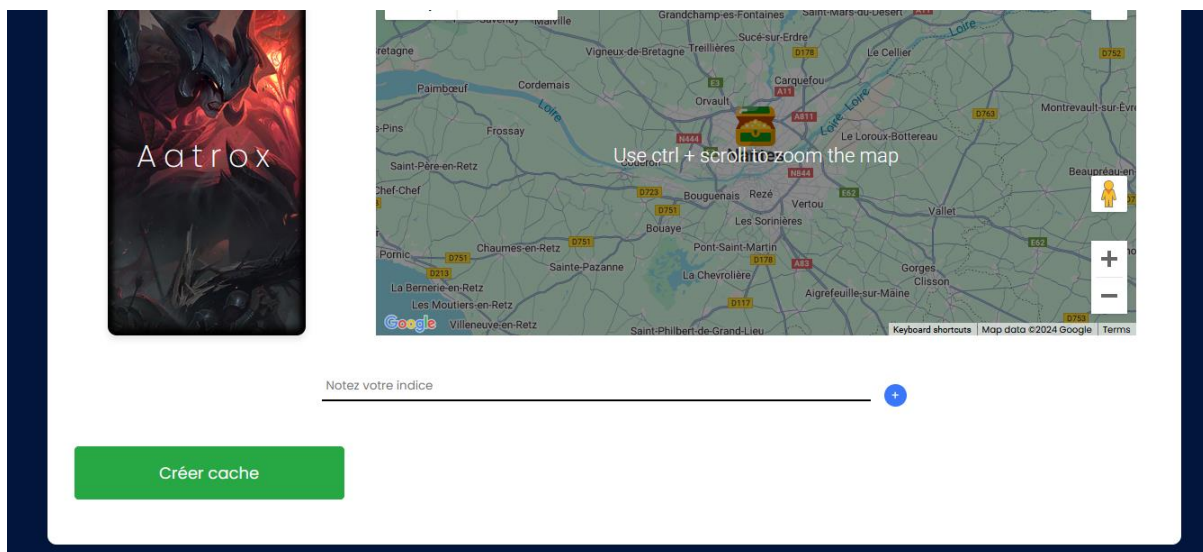
Cet écran s'affiche lorsque le joueur obtient une nouvelle carte après avoir remporté une partie. La carte obtenue est mise en avant avec son nom en grand, et un message en haut confirme son ajout à la collection. L'arrière-plan montre une carte géographique, ajoutant au contexte de la victoire.



5. Création d'une partie

Cet écran permet aux utilisateurs de créer une nouvelle partie de chasse. Ils peuvent sélectionner un emplacement sur la carte et fournir des indices pour guider les joueurs. Un

bouton « Créer cache » valide la création de la partie, ajoutant ainsi une nouvelle chasse à la liste des parties disponibles.



8. Problème rencontrés

- Spécifications des besoins étaient floues, menant à des réajustements.